

Módulo 3: Review de Arquitectura LLM

Embeddings y RAG

2026-04-30

Enlaces

-  segana@fen.uchile.cl
 -  <https://segana.netlify.app>
 -  <https://www.linkedin.com/in/sebastian-egana-santibanez/>
 -  <https://github.com/sebaegana>
-

Módulo 03

Clase 01 Review

Embeddings y RAG

Objetivo de la sesión

- Entender cómo se conectan embeddings y RAG
 - Traducir estos conceptos a decisiones de diseño
 - Revisar ejemplos aplicados a contextos organizacionales
 - Preparar base conceptual para agentes y soluciones con documentos
-

Hoja de ruta

1. embeddings: cómo representar significado
 2. RAG: cómo conectar documentos con generación
-

Idea central de la clase

Un LLM útil en una organización no depende solo del modelo.

También depende de:

- cómo representamos la información
 - cómo armamos el contexto
 - cómo recuperamos contenido relevante
-

Embeddings

¿Qué son?

Cada texto se transforma en una representación numérica.

Eso permite capturar relaciones como:

- significado
- similitud
- cercanía semántica
- contexto de uso

Idea simple

Los embeddings son una forma de traducir lenguaje a vectores comparables.

Una intuición más precisa

Un embedding no es “un número”, sino un vector:

- una lista larga de valores
- en muchas dimensiones
- donde cada posición captura patrones aprendidos

Clave conceptual

Textos parecidos en significado tienden a quedar cerca en el espacio vectorial.

¿Qué aprende el modelo?

Durante el entrenamiento, el sistema aprende regularidades del lenguaje:

- qué términos aparecen en contextos parecidos
- qué diferencias ayudan a distinguir significados
- qué relaciones semánticas aparecen de forma recurrente

Idea clave

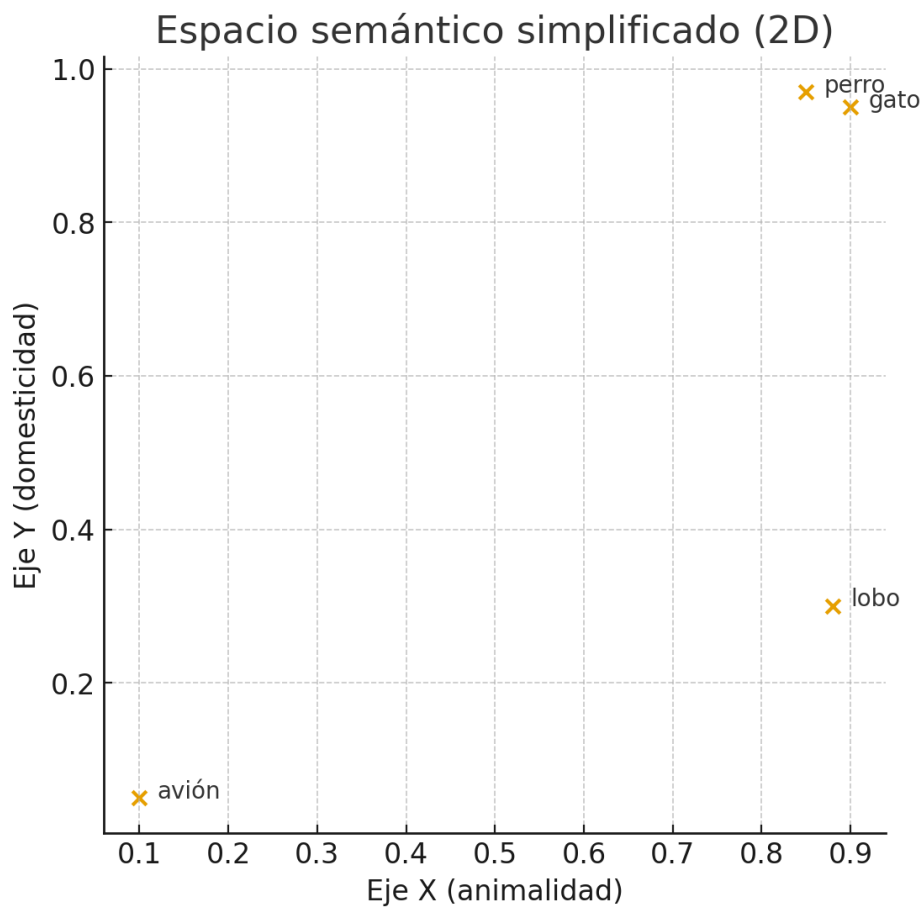
No memoriza definiciones como diccionario; aprende patrones de uso.

Ejemplo esquemático en 2 dimensiones

Imaginemos un caso muy simplificado donde solo tenemos dos ejes:

- eje X: animalidad
- eje Y: domesticidad

En la realidad hay muchas más dimensiones, pero esta simplificación sirve para la intuición.



Cómo leer ese esquema

En ese gráfico:

- gato y perro están cerca por similitud de uso
- lobo comparte parte del significado, pero no todo
- avión queda lejos porque pertenece a otra zona semántica

Idea clave

La distancia representa similitud aprendida desde el lenguaje.

¿Por qué importa esto en la práctica?

Porque permite comparar significado y no solo coincidencia literal.

Ejemplo

Un usuario pregunta:

“¿Qué ayuda puedo pedir cuando me jubile?”

Y el documento dice:

“beneficios disponibles al momento de pensionarse”

Con embeddings, ambos textos pueden quedar cerca aunque no usen las mismas palabras.

Embeddings y búsqueda semántica

Una aplicación clave es esta:

1. convertir documentos en embeddings
2. convertir la consulta en embedding
3. buscar los vectores más cercanos
4. recuperar los textos relevantes

Resultado

El sistema encuentra contenido por sentido y no solo por palabras exactas.

Ejemplo práctico: generar embeddings

```
from openai import OpenAI
import numpy as np
import faiss

client = OpenAI()

sentences = [
    "The dog is barking.",
    "A cat is sleeping on the couch.",
    "A car is driving down the street.",
    "I love my pet animal."
]

response = client.embeddings.create(
    model="text-embedding-3-small",
    input=sentences
)

vectors = np.array([data.embedding for data in response.data])
print("Dimensión de los embeddings:", vectors.shape)
```

Salida esperada:

```
Dimensión de los embeddings: (4, 1536)
```

Ejemplo práctico: buscar similitud

```
index = faiss.IndexFlatL2(vectors.shape[1])
index.add(vectors)

query = "My puppy is barking loudly."
query_vec = client.embeddings.create(
    model="text-embedding-3-small",
    input=query
).data[0].embedding

D, I = index.search(np.array([query_vec]), k=2)
print("Resultados similares:")
for idx in I[0]:
    print("-", sentences[idx])
```

Resultados similares:

- The dog is barking.

- I love my pet animal.
-

Lectura práctica

La consulta no es idéntica a los textos originales.

Sin embargo, el sistema igual recupera contenido relevante porque trabaja sobre vecindad semántica.

Idea clave

La similitud depende del significado compartido, no de repetir exactamente las mismas palabras.

RAG

Conexión con embeddings

En **Retrieval-Augmented Generation**, los embeddings ayudan a decidir qué fragmentos conviene traer al contexto del modelo.

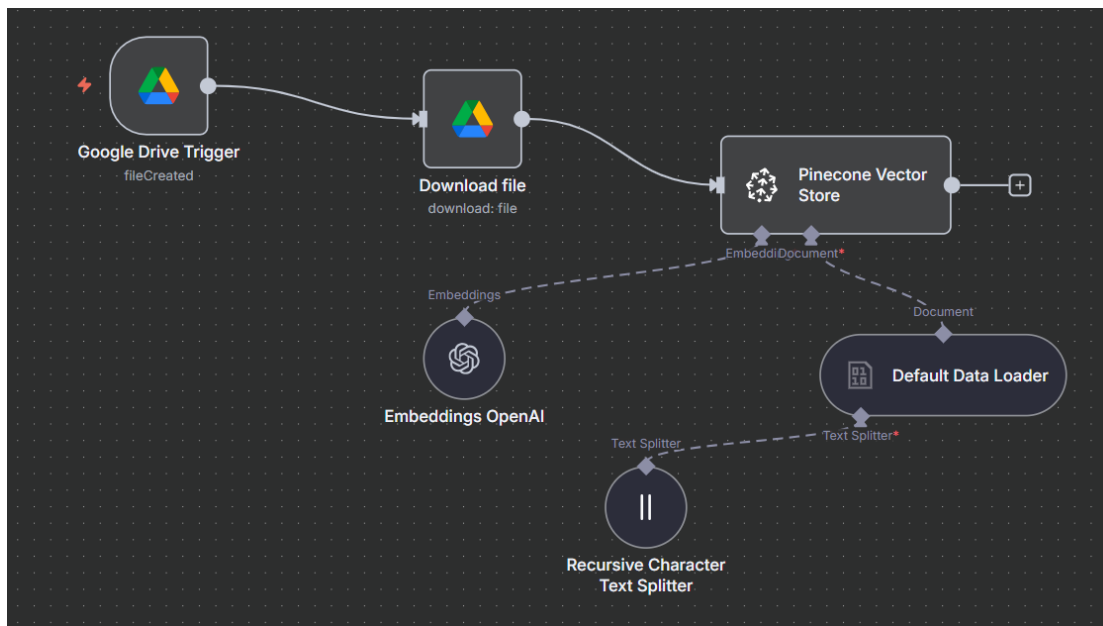
Flujo simple

- el usuario pregunta
- la consulta se vectoriza
- se buscan trozos similares en una base vectorial
- esos trozos se agregan al prompt
- el LLM responde usando ese contexto

Idea clave

Los embeddings no reemplazan al LLM: lo ayudan a encontrar mejor el contexto.

Ejemplo visual de flujo RAG



Lectura simple

La imagen muestra la etapa de ingestión: cómo un documento entra al sistema, se procesa y queda listo para ser recuperado después.

Qué hace cada nodo del flujo RAG

- Google Drive Trigger: detecta un archivo nuevo
- Download file: descarga el documento
- Default Data Loader: transforma el archivo en texto
- Recursive Character Text Splitter: divide en fragmentos
- Embeddings OpenAI: convierte cada fragmento en vector
- Pinecone Vector Store: guarda vectores y metadata

Idea clave

RAG no empieza respondiendo; primero prepara una base consultable.

Embeddings y base vectorial en un RAG

Cada fragmento se guarda junto con:

- el texto del chunk
- su embedding
- metadata como fuente, fecha o tema
- un identificador para volver al origen

Idea clave

La base vectorial guarda texto, representación semántica y contexto operativo.

Ejemplo aplicado

Imaginemos una base documental de una AFP con estos chunks:

- Chunk A: “Para pensionarse por vejez se requiere cumplir edad legal y requisitos vigentes.”
- Chunk B: “Los beneficiarios pueden revisar certificados en la sucursal virtual.”
- Chunk C: “Los canales de atención incluyen web, call center y oficina.”

Y metadata como:

- `area = pensiones`
- `fuentes = manual_beneficios_2026.pdf`
- `canal = web`

¿Para qué sirve?

Permite combinar similitud semántica con filtros útiles para el negocio.

Retrieval: cómo encuentra contexto

El retrieval es la etapa donde el sistema busca los fragmentos más útiles para responder.

Decisiones técnicas

- cuántos resultados traer (`top-k`)
- si usar filtros por metadata
- si exigir umbral mínimo de similitud
- si combinar búsqueda semántica con keywords

Idea clave

Recuperar bien no es solo buscar parecido, sino traer contexto suficiente y relevante.

Ejemplo: retrieval y priorización

Supongamos la consulta:

“¿Qué necesito para pensionarme y dónde hago el trámite?”

Un retrieval razonable debería priorizar fragmentos sobre:

- requisitos para pensionarse
- canales o lugares para iniciar el trámite
- documentación asociada

Y no debería poner arriba material menos útil, como certificados o reclamos no relacionados.

Reranking: segundo filtro de relevancia

Después del retrieval, algunos sistemas aplican reranking:

- primero recuperan varios candidatos
- luego un segundo modelo los reordena
- y dejan más arriba los fragmentos más útiles

Idea clave

Retrieval encuentra candidatos razonables; reranking mejora el orden final.

Límites y cuidados

RAG mejora el acceso a contexto, pero no resuelve todo por sí solo.

Los problemas más comunes son:

- documentos desactualizados
- chunking pobre
- mala metadata
- recuperación insuficiente o excesiva

Regla práctica

Buen modelo + mala base documental = mala respuesta igual.

Cierre

Cómo se conectan los conceptos

- `embeddings` representan significado
- RAG usa esa representación para recuperar contexto relevante

Idea final

Si falla una de estas piezas, cae la calidad de toda la solución.

Preguntas para discutir

1. ¿Qué pasa si el embedding es bueno, pero los documentos están mal fragmentados?
 2. ¿Qué pasa si recuperamos muchos chunks, pero el prompt está desordenado?
 3. ¿Qué decisiones de diseño son técnicas y cuáles son de negocio?
-

Cierre y transición

Esta clase deja la base para discutir:

- agentes
 - herramientas
 - memoria
 - arquitecturas aplicadas
-